Chapter 2: Review of Object Orientation

2.0 What is Object Orientation?

- Procedural abstraction organizes computation into procedures. Data abstraction groups information describing an entity.
- Object orientation places behaviour with the data it operates on. A running program consists of collaborating objects.
- Abstraction emphasizes relevant characteristics. Modularity divides code into manageable units. Encapsulation groups state and behaviour and supports information hiding.

2.1 Classes and Objects

- An object is an individual runtime entity with identity, state, and behaviour. A class describes its instances' common structure and behaviour.
- Film can be a class and a particular film an instance. Genre can be a class and Science Fiction one instance.
- Choose singular, descriptive class names such as BankAccount, with the right level of generality and one clear meaning.

2.2 Instance Variables

- Instance variables hold information for each object. Attributes hold simple values; associations represent relationships with other objects.
- A reference variable refers to an object; it is not the object itself. Multiple variables can refer to the same object, and a variable may later refer to another object.
- The variable's declared type determines which operations are available through it.

2.2.0 Class variables

- A static variable belongs to the class and is shared among its instances.
- Constants and lookup data are common uses. Mutable shared state needs care because a change affects all its users.

2.3 Methods, Operations, and Polymorphism

- An operation describes behaviour independently of its implementation. A method supplies a class's implementation.
- Polymorphism allows the same operation to behave differently for different kinds of objects; circles and rectangles calculate area differently.

- Polymorphic methods reduce repeated type tests and switch statements.

2.4 Inheritance Hierarchies

- A superclass defines common features. Subclasses inherit applicable features and may add or specialize behaviour.
- Apply the is-a rule: a checking account is an account, but a province is not a country.
- Every inherited feature must make sense in the subclass. A part-of relationship is an association rather than inheritance.

2.5 Inheritance, Polymorphism, and Variables

2.5.0 Abstract classes and overriding

- Declare an operation at the highest sensible level. An abstract method specifies behaviour without implementing it there.
- An abstract class cannot be instantiated. A concrete class must implement or inherit implementations of all required abstract methods.
- Overriding supplies a subclass implementation of an inherited instance method. It can extend or optimize behaviour while preserving the contract.
- Overloading uses different parameter lists and is distinct from overriding.

2.5.1 Dynamic binding

- The declared type controls whether a call is legal; the runtime object's class determines which overridden instance method executes.
- Look first for an implementation in the runtime class, then up its superclass chain.
- Example: Shape s = new Circle(); s.area(); executes the Circle implementation.
- Casting changes the type through which a reference is viewed, not the object's class. An invalid downcast can fail at runtime.

2.6 Java Fundamentals

2.6.0 Language and documentation

- The lecture traces object orientation through Simula, Smalltalk, and C++, then Java's development from Oak and its 1995 launch.
- Java compiles to bytecode executed by a virtual machine. Read class and method documentation to understand library behaviour.

- Javadoc generates API documentation from source declarations and comments.

2.6.1 Characters, strings, and collections

- char holds one UTF-16 code unit; Character is its wrapper class. Some Unicode characters require two code units.
- String represents text; + concatenates strings.
- Arrays have fixed length. ArrayList supports a resizable collection. Iterators traverse elements using hasNext() and next().
- Generics express element types: ArrayList<String> names = new ArrayList<>(); avoids many explicit casts during retrieval.

2.6.2 Exceptions and interfaces

- Exceptions signal failures. A try/catch block handles specified types of exceptions; the handling should reflect the actual problem.
- An interface specifies behaviour implementing classes promise to provide. A class can implement multiple interfaces, and a variable can have an interface type.
- Runnable, Collection, Iterator, and Comparable are library examples.
- The slides' prohibition on implementation code in interfaces describes older Java. Modern Java permits default and static interface methods.

2.6.3 Packages and access

- Packages group related classes and provide namespaces. Imports allow short type names in source code.
- public permits broad access; private hides implementation within the class and its enclosing nest; no modifier gives package access.
- protected permits package access and subclass access, subject to Java's additional rules across packages.
- Use the narrowest practical visibility.

2.6.4 Threads

- A thread is an execution sequence that can progress concurrently with other threads.
- Implement Runnable.run(), create a Thread using that Runnable, and call start(). Alternatively, subclass Thread.
- Calling run() directly uses the calling thread; start() begins a separate thread of execution.

- Coordinate access when threads share mutable state.

2.7 Programming Style

- Prefer understandable code and descriptive names. Shorter or cleverer code is not automatically better.
- Explain non-obvious decisions in comments and organize class elements consistently.
- Extract shared behaviour into methods instead of copying code, so fixes do not need to be repeated.
- Follow the is-a rule, hide implementation details, and separate UI code from domain logic to improve reuse.

2.8 Difficulties and Risks

- Languages and libraries evolve. Deprecated features may need replacement during maintenance.
- Measure efficiency against the application's requirements. Virtual-machine execution and dynamic binding alone do not establish that a program is too slow.